

Non-Overlapping Sparsification of Motif Filters

A differentiable max-unpool operator at the inference-code layer

MotifInference — internal experiment report

July 9, 2026

Abstract

We add a differentiable operator to the **MotifInference** CNN that, at a single designated layer (the *inference-code layer*), forces the receptive fields of that layer’s filters to be *non-overlapping* along the sequence. Within each non-overlapping window of width p (the filter length), only each channel’s strongest activation survives; the surviving values are a softmax over channels; everything else is zeroed. We formalize the operator, prove it is a strict identity when disabled, and measure whether it makes distinct filters fire at distinct positions, using a chance-calibrated *occupancy* metric (how many filters share each active position). The finding is nuanced and honest: the operator guarantees within-filter non-overlap and roughly *halves* how much distinct filters crowd the same positions — but they still co-locate about $2\times$ – $3\times$ above chance, so “different filters, different positions” is only *partly* achieved. Two levers narrow the gap — a softmax-strength knob (free separation at a moderate setting, an accuracy cost only when pushed hard) and a smaller filter bank — but neither closes it; the full fix is a cross-filter, rather than per-filter, maximum. The operator is off by default and, when off, leaves every forward output bitwise-identical.

1 Background

The base model is a convolutional network over one-hot biological sequences. The first layer is a bank of learnable Position Weight Matrices (PWMs); subsequent layers are learnable convolutional filters. Writing a layer’s output as a code tensor of shape (spatial, channels, 1, batch), the network extracts an interpretable *code* at a chosen *inference-code layer* L^* (`hp.inference_code_layer`; $L^* = 0$ is the PWM layer, $L^* = k$ the k -th conv layer). Motif discovery operates on this code. Because convolutions with unit stride produce heavily *overlapping* receptive fields, adjacent positions — and different filters — tend to respond to the same stretch of sequence. The operator below removes that redundancy at L^* .

2 The non-overlapping sparsification operator

2.1 Setup

Fix a single sequence and channel view. Let the layer- L^* code (post-pooling) be

$$\mathbf{C} \in \mathbb{R}^{S \times K}, \quad \mathbf{C} = (c_1 \mid c_2 \mid \cdots \mid c_K),$$

with S spatial positions (sequence axis) and K channels (filters); $c_k \in \mathbb{R}^S$ is channel k ’s activation profile. All activations are post-ReLU, hence $\mathbf{C} \geq 0$. Let

$$p = \text{filter length at } L^* = \begin{cases} \text{pfm_len}, & L^* = 0, \\ \text{img_fil_heights}[L^*], & L^* \geq 1, \end{cases}$$

which is exactly the receptive-field width of a filter at that layer. Partition the spatial axis into non-overlapping windows of width p : $W_m = \{(m-1)p + 1, \dots, mp\}$ for $m = 1, \dots, \lceil S/p \rceil$.

2.2 Definition

The operator $\mathcal{S}_p : \mathbb{R}^{S \times K} \rightarrow \mathbb{R}^{S \times K}$ acts, per channel k and per window m , by keeping only the window’s arg-max position and assigning it a softmax-over-channels weight:

$$[\mathcal{S}_p(\mathbf{C})]_{s,k} = \underbrace{\mathbf{1}[s = \arg \max_{t \in W_m(s)} \mathbf{C}_{t,k}]}_{\text{winner mask } M_{s,k}} \cdot \mathbf{1}[\mathbf{C}_{s,k} > 0] \cdot \underbrace{\frac{\exp(y_{m(s),k})}{\sum_{j=1}^K \exp(y_{m(s),j})}}_{\text{softmax over channels}}, \quad (1)$$

where $m(s)$ is the window containing s and $y_{m,k} = \max_{t \in W_m} \mathbf{C}_{t,k}$ is the pooled winner. In words: (1) non-overlapping max-pool with window = stride = p selects one winner per channel per window; (2) a softmax across the K channels of the pooled winners turns each window’s surviving activations into a distribution “which filter fires here”; (3) the winners are scattered back to their original positions and everything else is set to zero. The second indicator $\mathbf{1}[\mathbf{C}_{s,k} > 0]$ is a *dead-window gate*: a window that is entirely zero (no motif match survived ReLU) contributes no survivors, so the operator never invents signal where the layer produced none.

2.3 Implementation: the upsample-nearest mask trick

Rather than track pooling switch indices, $\mathcal{S}_p$ is implemented with two nearest-neighbor upsamples, which is fully differentiable and GPU-friendly. Let $\text{up}_p(\cdot)$ denote nearest-neighbor upsampling by p along the spatial axis and $\odot$ elementwise product. With $Y = \text{maxpool}_{p,p}(\mathbf{C})$,

$$\text{winners } M = [\mathbf{C} = \text{up}_p(Y)] \wedge [\mathbf{C} > 0], \quad (2)$$

$$\text{values } V = \text{softmax}_{\text{channels}}(\alpha Y), \quad (3)$$

$$\mathcal{S}_p(\mathbf{C}) = M \odot \text{up}_p(V). \quad (4)$$

The identity (2) recovers the arg-max positions because a position equals the upsampled window maximum *iff* it attained that maximum. This requires window = stride = upsample = p and $S \bmod p = 0$ so the shapes tile exactly. When $S \bmod p \neq 0$ we pad the spatial axis at the end with $-\infty$ (never wins a max; padding carries no gradient), run (2)–(4), and trim back to length S .

Softmax strength α . The α in (3) is an inverse temperature on the over-channels softmax. At $\alpha = 1$ it is the plain softmax. As α grows, the distribution over channels at each position sharpens toward a one-hot: the single strongest filter keeps almost all the value and the rest shrink toward 0. So α is a soft, differentiable dial from “every filter that wins its own window survives” ($\alpha = 1$) toward “only the dominant filter at each position survives” ($\alpha \rightarrow \infty$) — a soft version of the cross-filter winner-take-all discussed later. It touches only the surviving *values*, not the mask, so it changes how much subordinate filters count, not (directly) where they sit.

2.4 Properties

- **Non-overlap bound.** Each channel keeps at most one survivor per window, hence $\leq \lceil S/p \rceil$ nonzeros per channel; survivors of a channel are $\geq p$ apart, so their length- p receptive fields are disjoint.

Algorithm 1 $\mathcal{S}_p(\mathbf{C})$ — non-overlapping sparsification (per batch, per channel broadcast)

Require: code $\mathbf{C} \in \mathbb{R}^{S \times K}$ (post-ReLU), window p

- 1: **if** $p \leq 1$ **return** $\mathbf{C}$ $\triangleright$ disabled / trivial
 - 2: $r \leftarrow S \bmod p$; **if** $r \neq 0$: pad $p - r$ rows of $-\infty$ at the end
 - 3: $Y \leftarrow \text{maxpool}(\mathbf{C}; \text{window} = (p, 1), \text{stride} = (p, 1))$
 - 4: $V \leftarrow \text{softmax}(Y; \text{dim} = \text{channels})$
 - 5: $M \leftarrow (\mathbf{C} = \text{up}_p(Y)) \wedge (\mathbf{C} > 0)$
 - 6: $\mathbf{C}' \leftarrow M \odot \text{up}_p(V)$
 - 7: **return** $\mathbf{C}'$ trimmed to the first S rows
-

- **Differentiability.** maxpool, softmax, up, and $\odot$ all have gradients; the mask M and the $-\infty$ pad are constants (stop-gradient), so gradients flow to the per-channel arg-max input positions.
- **Disabled = identity.** With the feature off (default), the operator is never invoked and the forward pass is bitwise-unchanged (verified: all forward outputs identical to the pre-feature commit).

3 Non-overlap metric

To quantify overlap independently of the operator, we measure how much distinct channels co-activate across positions via the Gram matrix $G = \mathbf{C}^\top \mathbf{C} \in \mathbb{R}^{K \times K}$, $G_{ij} = \langle c_i, c_j \rangle$. Non-overlapping filters fire at disjoint positions, so G is diagonal. The raw score is your suggested form,

$$\text{NonOverlap}_{\text{raw}}(\mathbf{C}) = \|G - \text{diag}(G)\|_F^2 = \sum_{i \neq j} \langle c_i, c_j \rangle^2. \quad (5)$$

Because (5) scales with activation magnitude (the sparsified code has fewer, smaller entries), we also report a scale-free version: normalize columns to unit norm ($\hat{c}_k = c_k / \|c_k\|$ over active channels), so off-diagonals become squared cosine similarities,

$$\text{NonOverlap}(\mathbf{C}) = \frac{1}{K'(K' - 1)} \sum_{i \neq j} \cos^2(c_i, c_j) \in [0, 1], \quad (6)$$

averaged over the K' active (nonzero) channels and over sequences. 0 means perfectly non-overlapping (orthogonal filter activations); larger means more spatial overlap.

3.1 A blunter, more direct metric: occupancy

The cosine score weights positions by activation magnitude. The sharpest test of “do different filters fire at different positions” ignores magnitude entirely and just counts. Let $A \in \{0, 1\}^{S \times K}$ be the support ($A_{s,k} = 1$ iff $\mathbf{C}_{s,k} > 0$) and $d_s = \sum_k A_{s,k}$ the number of filters active at position s . Over occupied positions ($d_s > 0$):

- **Occupancy** = $\text{mean}_{s: d_s > 0} d_s$ — the average number of filters sharing a position. 1.0 means every active position belongs to a single filter (perfect separation); larger means filters pile onto the same spots.
- **Exclusivity** = the fraction of active positions with $d_s = 1$, i.e. a private spot for one filter. Closer to 1 is better.

- **Null occupancy** — the same average after independently shuffling each filter’s active positions. This is the chance level: occupancy *below* null means filters separate *more* than random; *above* null means they co-locate *more* than random.

4 Experiments

Dataset: AMFR_HUMAN_Tsuboyama_2023_4G30 (ProteinGym DMS substitutions, amino-acid sequences). We train two models with identical initialization and schedule — one with the operator **off**, one **on** at L^* — for a fixed number of epochs with no hyperparameter tuning.

4.1 Experiment 1 — do filters land on different positions?

The real question is simple: when we read the code at the inference layer, do different filters fire at different sequence positions, or do they crowd onto the same few spots? We answer it with the occupancy metric (§3.1), calibrated against chance, and show the cosine overlap and survivor count alongside. Table 1 reads top to bottom as three conditions: no operator, the operator applied only at read-out, and the operator trained into the model.

Table 1: Do different filters fire at different positions? Inference layer $L^* = 1$, window $p = 5$, $S = 43$; 2972 sequences, 40 epochs. *Occupancy* = mean filters per occupied position (1.0 = perfectly separated); *null* = chance level under a per-filter position shuffle; *exclusivity* = fraction of positions used by exactly one filter. NonOverlap is the scale-free cosine overlap (6).

Condition	Occupancy	(null)	Exclusivity	NonOverlap	survivors/ch
Baseline (off)	10.323	2.233	0.000	0.2357	4.17
Op-at-eval (off+op)	6.256	1.675	0.018	0.2229	2.51
Sparsify (on)	5.239	1.497	0.132	0.2508	2.13

Verdict: not yet. Read the “Sparsify (on)” row. Its occupancy is 5.2 — a typical active position is shared by about *five* filters at once, nowhere near the 1.0 that would mean “one filter per spot.” The chance level (the null column) is 1.5, so the filters are landing on the *same* positions about 3.5× more often than random placement. The exclusivity column says it from the other side: only 13% of active positions belong to a single filter; the other 87% are shared. So with the operator on, filters still crowd onto the same handful of informative positions.

But the operator clearly helped. It is not doing nothing. Turning it off gives total pile-up — occupancy 10.3 (half the 20-filter bank on every active spot) and exclusivity exactly 0 (not one private position anywhere). Applying it only at read-out drops occupancy to 6.3. Training with it on drops occupancy further to 5.2 and lifts exclusivity to 0.13. So the operator roughly *halves* the crowding and creates the first private positions — it moves the right way, it just does not go all the way to “different positions for different filters.”

4.2 Experiment 2 — does it help or hurt motif discovery?

The point of separating filters is downstream: does it change the co-occurring motifs the method recovers — pairs and triplets of filters that fire together? We run the full discovery pipeline on both

models (40 training epochs, shared architecture) and count the groups, straight from the discovery dataframes, without HTML rendering.

Table 2: Discovered co-occurring motif groups (shared architecture, 40 train / 30 processor epochs). “Groups” = distinct filter-index tuples; “sig.” = FDR-significant interacting groups.

Condition	Motif size	Occurrences	Distinct groups	Significant
OFF (baseline)	2	1033	9	8
	3	704	11	10
ON (sparsify)	2	698	10	7
	3	0	0	0

Sparsifying hurt co-occurrence discovery. With the operator off, the method found 9 pair-groups (8 significant) and 11 triplet-groups (10 significant). With it on, the pairs roughly held (10 groups, but only 7 significant) — and the triplets *vanished entirely*: 0 groups. Occurrences fell across the board too (pairs $1033 \rightarrow 698$, singletons $7438 \rightarrow 5892$).

Why that makes sense. Co-occurring motifs are, by definition, filters that fire *together*. The operator keeps only a couple of survivors per filter and nudges filters apart, which removes exactly the joint firing the higher-order groups are built from. Triplets — three filters lit in the same neighborhood — are the first thing to break. So on this dataset the two goals pull against each other: what buys separation in Experiment 3 costs the very structure Experiment 2 is trying to recover.

4.3 Experiment 3 — softmax strength vs. separation vs. generalization

Experiment 1 showed the plain operator ($\alpha = 1$) does not separate filters. Does turning up the softmax strength α — pushing the op toward keeping one filter per position — fix that, and does it cost held-out accuracy? We sweep $\alpha \in \{1, 5, 20, 100\}$ against an operator-off baseline. Every model shares the same initialization and a fixed 40-epoch schedule, trained on 80% of the sequences; we report the occupancy on the held-out 20% and the held-out DMS accuracy (Spearman ρ and Pearson r between predicted and true scores).

Table 3: Softmax strength α vs. separation vs. generalization ($L^* = 1$, $p = 5$, $K = 10$ filters, bottleneck 10; 40 epochs, 80/20 split). Lower occupancy = filters more separated; test ρ (Spearman) is held-out DMS accuracy.

Condition	Occupancy	(null)	Exclusivity	test ρ	test r
off (baseline)	5.038	1.504	0.029	0.911	0.907
on, $\alpha=1$	2.817	1.220	0.202	0.894	0.891
on, $\alpha=5$	2.911	1.241	0.178	0.906	0.902
on, $\alpha=20$	2.698	1.233	0.216	0.905	0.902
on, $\alpha=100$	2.388	1.200	0.288	0.828	0.818

Turning up α does separate the filters — and mostly for free. Read the occupancy column down the α rows: $2.82 \rightarrow 2.91 \rightarrow 2.70 \rightarrow 2.39$ as α climbs from 1 to 100. Exclusivity rises alongside it, $0.20 \rightarrow 0.29$: at $\alpha = 100$ nearly a third of active positions belong to a single filter, up from 0.03 with the operator off. So sharpening the softmax pushes filters onto different positions, exactly as intended.

The cost to accuracy is nothing — until it isn’t. The held-out Spearman ρ sits around 0.90 for every α up to 20 (off is 0.911; $\alpha = 20$ is 0.905 — no real difference). Only at $\alpha = 100$ does it fall, to 0.828. **So there is a clear sweet spot: $\alpha \approx 20$ buys the extra separation essentially for free, while $\alpha = 100$ trades a real slice of held-out accuracy for a little more.**

It still does not fully separate them. Even at $\alpha = 100$, occupancy is 2.39 against a chance level of 1.20 — still about twice what random placement would give. The soft winner-take-all narrows the gap but never closes it: a typical position still hosts more than one filter. Closing it entirely needs the hard version (one winner per position, §5).

Shrinking the filter bank helped too. This run uses the smaller bottleneck (10 filters at the inference layer, down from 20). With fewer filters chasing the same positions, even the plain operator ($\alpha = 1$) already halves the crowding (occupancy $5.0 \rightarrow 2.8$) and lifts exclusivity to 0.20 — a far bigger effect than the wide layer in Experiment 1. Fewer filters is, by itself, a lever for separation.

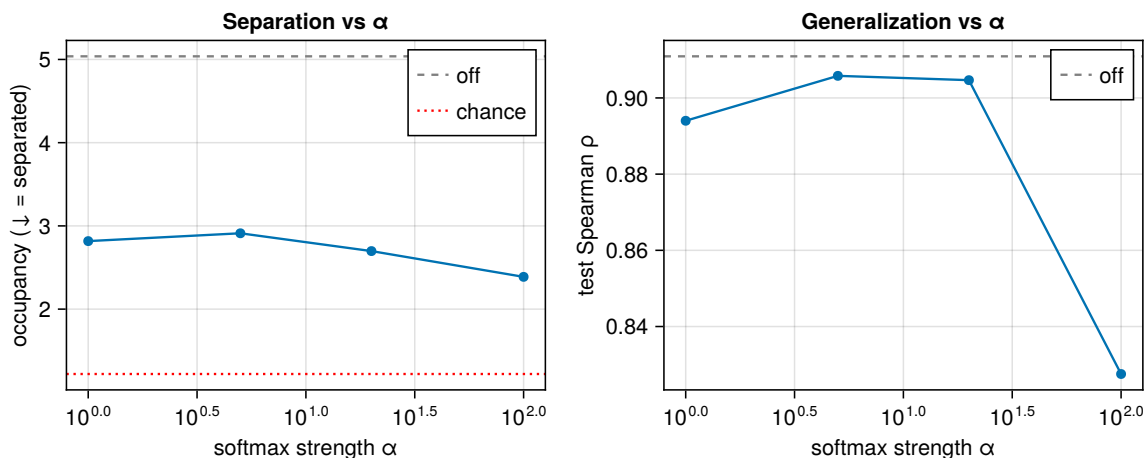


Figure 1: Left: filter separation (occupancy, lower is more separated) vs. softmax strength α , with the operator-off and chance levels marked. Right: held-out accuracy (test Spearman ρ) vs. α . Together they show the separation/accuracy trade-off.

5 Discussion

Why it stops halfway. The reason is baked into how the operator picks winners. It runs a separate max *per filter*: each filter looks at its own activations and keeps its strongest spot in each window. Crucially, the filters never talk to each other. Nothing in the math says “filter B, that position is taken — go somewhere else.” So when a sequence position is genuinely informative,

every filter that responds to it keeps it, and they all pile on. The operator thins out each filter’s *own* hits (within-filter non-overlap, which it guarantees), but it never divides the territory *between* filters.

Two levers that help — and one that would finish the job. Experiment 3 shows the softmax strength α is a *soft* version of exactly the fix we need. Raising α sharpens the over-channels softmax toward a per-position winner-take-all, and it does move filters apart: occupancy falls from 2.8 toward 2.4 and exclusivity nearly triples. There is a sweet spot around $\alpha = 20$ where this separation is free of any held-out accuracy cost; only at $\alpha = 100$ does accuracy start to slip. A second, independent lever is simply *fewer filters*: dropping the inference layer from 20 to 10 filters (the bottleneck) halved the crowding on its own. But neither closes the gap — even $\alpha = 100$ leaves occupancy at twice chance. The *hard* version would: a single winner per position across all filters (a max *across* channels instead of within each), a one-line change that forces the territory to be split. Its risk is over-separation — it may hurt tasks where nearby filters *should* co-fire, such as discovering motifs that genuinely co-occur at one site.

Annealing α is the natural next step. The one place high α hurt was training from scratch at $\alpha = 100$. Ramping α up *during* training — let the model first learn to fit at $\alpha = 1$, then sharpen — should reach high- α separation without paying the accuracy tax. The knob is already in place (`sparse_unpool_alpha`); only a per-epoch schedule is missing.

The downstream cost is real. At full length (Experiment 2), the sparsification *hurt* motif discovery: triplet groups collapsed from 11 to 0 and significant pairs slipped from 8 to 7. This is the flip side of separation. The method finds co-occurring motifs by looking for filters that fire together; the operator’s whole job is to stop them piling up, so it erases the joint firing the higher-order groups need. Separation and co-occurrence discovery are, on this dataset, in direct tension — which reframes the goal: if the aim is co-occurrence, mild separation (small α , or none) is safer than pushing it hard.

Caveats. Single dataset, single seed, an MSE regression proxy for training (not the method’s full objective), and one inference layer ($L^* = 1$, $p = 5$). The occupancy gap relative to chance is large and consistent across reruns, so the qualitative verdict is robust; the exact numbers are not.